

Identity Share Discovery Design

Identity Management + Network-Share Discovery — Design & Survey

Revision: 1 **Last modified:** 2026-06-26T00:00:00Z **Status:** DRAFT (design + survey pass — no implementation) **Authority:** Operator capability request 2026-06-26; constitution §11.4.74 (catalogue-first), §11.4.150 (deep multi-angle research), §11.4.10 (credentials), §11.4.58 (PWU), §11.4.169 (test-type coverage), §11.4.162 (OpenDesign) **Scope:** catalog-api (Go backend) + catalog-web + catalogizer-desktop + catalogizer-android + catalogizer-androidtv + installer-wizard **Classification note:** This is a project-specific instantiation; the reusable pieces live in owned submodules (§11.4.28/§11.4.74).

⚠ This document references credential material by **env-var NAME only** (never a value), per §11.4.10. No secret value appears anywhere below.

0. Capability (verbatim operator intent)

“We provide identities; the System scans the network and all interfaces and protocols we support for available hosts and shares; each share is accessed first with NO credentials (anonymous), then with every added identity until one passes/logs in. We remember the (share, identity) combinations that work together. Extend the System for proper IDENTITY MANAGEMENT — an identity can be credentials (username+password), an API token, or other common auth means; support them ALL. Then extend the catalog with all detected network shares, track all duplicates, group everything so the user can easily browse, search and filter. Proper UI/UX and System support MUST exist.”

Decomposed into five capability pillars:

1. **Identity management** — extensible, multi-scheme identity store (credentials / api_token / ssh_key / ...).
 2. **Network discovery** — enumerate hosts across every interface + protocol, then enumerate shares per host.
 3. **Anonymous-first auth probing** — try each share anonymous, then each identity in order until one authenticates; **remember the working (share, identity, protocol) binding**.
 4. **Catalog extension** — auto-register every discovered share as a source, ingest media, track duplicates, group for browse/search/filter.
 5. **UI/UX** — identity-manager screen + discovered-shares screen + grouped/deduped browse, per app, OpenDesign-tokened light+dark.
-

1. Current-State Survey (§11.4.74, file:line evidence)

1.1 How shares are discovered / scanned today

Capability	Where	Verdict
Host discovery (announce/respond)	catalog-api/main.go:771-795 — digital.vasic.discovery/pkg/broadcast Announcer + Responder (UDP multicast, namespace catalogizer). This announces the catalog-api service , it does NOT scan for NAS hosts.	Service-presence only — not host/share inventory
TCP reachability probe	catalog-api/main.go:85-95 — digital.vasic.containers/pkg/discovery.NewTCPDiscoverer() probes a single configured target.	Point probe, not a sweep
Network host+SMB scanner (CIDR sweep)	submodules/discovery/pkg/scanner (Scanner iface: Scan/ScanHost) + submodules/discovery/pkg/smb/scanner.go (ports 445/139, concurrent, semaphore-limited, CIDR expansion).	EXISTS but UNWIRED into catalog-api — the reuse anchor for host/share sweep

Capability	Where	Verdict
SMB share enumeration on a host	catalog-api/internal/services/ smb_discovery.go:57-95 DiscoverShares() — connects 445, NTLM session, enumerateShares() mounts known share names.	Works, but guessing-based (see below)
SMB connection test	internal/services/ smb_discovery.go:178-225 TestConnection() — dials, mounts share, ReadDir(".").	Solid positive-evidence probe
Existing discovery HTTP routes	catalog-api/ main.go:1267-1271 — POST/GET /api/v1/.../smb/discover, /smb/test, /smb/browse → handlers.SMBDiscoveryHandler.	SMB-only, operator-supplied creds, no anon-first, no multi-identity, no remembering
Scan engine (per protocol)	internal/services/ universal_scanner.go:19-67 — UniversalScanner with ProtocolScanner registry + filesystem.ClientFactory; ScanJob keyed on *models.StorageRoot.	Generic, protocol-pluggable — the ingest target

Critical anti-bluff finding (§11.4.6 / §11.4.107):

smb_discovery.go:140-175 getCommonShares() returns a **hard-coded list of guessed share names** (shared, public, media, music,...) as a fallback “when host unreachable” (line 169-172 returns them even when len(accessibleShares)==0). This is a guessing path that can surface non-existent shares as discovered. The new pipeline **MUST** replace guess-fallback with honest “host unreachable / no shares enumerable → SKIP-with-reason” (§11.4.3) and only ever report a share proven mountable.

1.2 Protocols actually supported today

catalog-api/filesystem/interface.go:1-16 aliases
digital.vasic.filesystem/pkg/client.filesystem/factory.go:15-50+
CreateClient() switches on config.Protocol:

- **smb** (445, go-smb2 NTLM) — factory.go:18-27, low-level in catalog-api/filesystem/smb_client.go:38-73
- **ftp** (21, jlaffaye/ftp) — factory.go:29-37

- **nfs** (v3, build-tag gated `nfs_client_darwin.go/_windows.go`) — `factory.go:39–50`
- **webdav** (`catalog-api/filesystem/webdav_client.go`) — `factory` continues past line 50
- **local** (`path-traversal guarded`)

So today the supported protocol set is **SMB, FTP, NFS, WebDAV, local**. Only **SMB** has a *discovery/probe* path (`smb_discovery.go`). FTP/NFS/WebDAV/local have working *clients* but no *discovery/probe* surface.

1.3 How authentication is handled for a share/source today

- **Model:** `catalog-api/models/file.go:48–70`
`StorageRoot{ ... Username *string; Password *string; Domain *string ... }` — **credentials are embedded per share record**, plaintext.
- **DB:** `storage_roots` table — `database/migrations/000001_initial_schema.sqlite.up.sql:4–26 + ...up.sql:5–27`. Columns `username TEXT, password TEXT, domain TEXT` stored **plaintext** (§11.4.10 gap).
- **Read path:** `handlers/scan_handler.go:294–308` `loadStorageRoot()`
`SELECT ... username, password, domain ...; root-set query`
`main.go:377–416` (`SELECT ... username, password ... FROM storage_roots WHERE protocol='smb' AND enabled=1`).
- **Client construction:** `handlers/copy.go:326` `createSmbClient()`, `factory.go:18–27` build the client straight from the per-root username/password.
- **No identity abstraction:** credentials cannot be shared across shares, cannot be rotated centrally, are not anonymous-first, and there is no “try identities in order” logic anywhere.

1.4 The catalog data model (source media)

- **Source = storage_roots** (`models/file.go:48–70`). One row per configured share/root.
- **Files → source FK:** `files.storage_root_id` references `storage_roots(id)` (`migrations/000001_initial_schema.up.sql:55`, `index idx_files_storage_root_path` line 109).
- **Scan history → source FK:** `scan_history.storage_root_id` (`...up.sql:105`, `index` line 114).
- **Aggregation to entities:** `internal/services/aggregation_service.go:60–67` `AggregateAfterScan(storageRootID)` walks media-bearing leaf files → `MediaItem` entities at **title granularity** (DEFECT-E fix, `aggregation_service.go:60–66` comment); repos `MediaItemRepository`, `MediaFileRepository`, `DirectoryAnalysisRepository`, `ExternalMetadataRepository`.

1.5 Existing duplicate-tracking + grouping (reuse, don't reinvent)

- **Dedup engine:** internal/services/duplicate_detection_service.go:15–90 — DuplicateDetectionService, DuplicateGroup{PrimaryItem, DuplicateItems, Confidence, DetectionMethod, MatchTypes, Status}, DuplicateItem{FileHash, FileSize, ExternalIDs, Fingerprints, ...}, similarity stack (SimilarityAnalysis, TextSimilarityMetrics: Levenshtein / JaroWinkler / Cosine / Jaccard / LCS / Soundex / Metaphone). **This already solves “same content across shares”.**
- **Grouping for browse:** aggregation_service.go already collapses files into title-level MediaItems. Multi-source dedup must feed these so one title with copies on 3 shares shows once with 3 origins.

1.6 App UIs for source/share config + login (current)

App	Stack / tokens	Source/share config UI
installer-wizard	Tauri2 + React + Tailwind/shadcn	src/components/wizard/SMBConfigurationStep.tsx:26–478 (host/port/share/user/pwd/domain + Test), NFSConfigurationStep.tsx:23–395; contexts/ConfigurationContext.tsx already models ConfigurationAccess{type: 'credentials', account, secret} separate from ConfigurationSource{type, url, access} — the closest existing identity? source split
catalogizer-desktop	Tauri2 + React + Tailwind	src/pages/SettingsPage.tsx:24–114 SMB config list (add/delete) + server URL test
catalog-web	React18 + Tailwind/shadcn + React Query + Zustand	src/pages/Settings.tsx:34–101 scans + storage-root selector; lib/api.ts exposes smbApi, scansApi, syncApi
catalogizer-androidtv	Compose-for-TV + Material3, OpenDesign-aligned (ui/theme/Theme.kt)	none (login + server discovery only ui/screens/login/LoginScreen.kt:224–270, 538–583)
catalogizer-android	Compose + Material3	none

Gaps: No app has a dedicated **identity manager**. OpenDesign tokens are referenced **only** in Android TV (§11.4.162 gap for web/desktop/phone). The installer-wizard’s Access²Source separation is the conceptual seed to lift backend-side.

1.7 .env identity keys — defined but UNWIRED

.env.example:46–65 defines (NAMES only): CATALOGIZER_IDENTITY_COUNT, CATALOGIZER_IDENTITY_<N>_TYPE, _USERNAME, _PASSWORD, _TOKEN, _KEY_PATH, _PASSPHRASE. **grep CATALOGIZER_IDENTITY over *.go = ZERO hits**. The identity contract exists only as documentation; **no code consumes it**. .env is gitignored (.gitignore:45–48, 319–320, 328). This is the central greenfield wiring gap.

1.8 §11.4.74 Catalogue-Check — reusable owned submodules

Submodule (module path)	Provides	Verdict
submodules/discovery (digital.vasic.discovery)	pkg/scanner host sweep, pkg/smb SMB scanner (445/139, CIDR), pkg/broadcast mDNS/UDP, pkg/report	REUSE + EXTEND (add NFS/FTP/WebDAV scanners via Scanner iface)
submodules/auth (digital.vasic.auth)	pkg/jwt, pkg/apikey (KeyStore iface), pkg/oauth (CredentialReader/TokenRefresher/AutoRefresher), pkg/token (Store)	REUSE as the identity credential-scheme backbone; EXTEND with ssh_key + raw-credentials schemes
submodules/filesystem (digital.vasic.filesystem)	pkg/client 5-protocol clients + pkg/factory	REUSE (already aliased by catalog-api)
submodules/security (digital.vasic.security)	pkg/securestorage (encrypted at-rest), pkg/pii redactor, pkg/e2ee	EXTEND — encrypt identity secrets at rest + redact hosts/users in logs
submodules/database (digital.vasic.database)	pkg/repository[T], sqlite/postgres adapters, pkg/migration	REUSE for new tables

Submodule (module path)	Provides	Verdict
submodules/config (digital.vasic.config)	pkg/env struct-tag env loader	REUSE to parse CATALOGIZER_IDENTITY_*
submodules/media, submodules/watcher, submodules/entities	metadata extraction / incremental watch / shared models	EXTEND (post-discovery ingest)
submodules/challenges, submodules/helix_qa	Challenge bank + autonomous QA	REUSE for §11.4.169 coverage

Conclusion: ~80%+ of the building blocks exist. The new work is **composition + the missing identity/probe/remember logic + the 5 app UIs**, NOT new infrastructure.

2. Proposed Design

2.1 Identity model (extensible, multi-scheme)

A first-class **Identity** decoupled from any share:

```
Identity {
    id          int64
    name        string           // operator-facing label, e.g.
    "nas-admin"
    type        string           // credentials | api_token |
    ssh_key | webdav_basic | oauth2 | anonymous(synthetic)
    // scheme-specific fields, encrypted at rest (security/pkg/
    securestorage):
    username    *string
    secret_ref  *string           // opaque handle into the secret
    store; NEVER the raw secret in this row
    domain      *string           // SMB
    key_path    *string           // ssh_key
    token_ref   *string           // api_token (handle, not the
    token)
    enabled     bool
    priority    int               // order to try during probing
    (lower = first)
    created_at / updated_at
}
```

- **Pluggable AuthScheme interface** (Go): Probe(ctx, client filesystem.FileSystemClient, share ShareTarget, id Identity) (ok bool, evidence ProbeEvidence, err error). One implementation per type. New scheme = new struct registered in a registry (open/closed).

This mirrors `universal_scanner.go`'s `ProtocolScanner` registry pattern (`universal_scanner.go`:64–67) for consistency.

- **Loader sources (precedence, §11.4.6 deterministic):** (1) `CATALOGIZER_IDENTITY_*` env via `digital.vasic.config/pkg/env` → seeds the identity table on boot; (2) identities added via UI/API at runtime; (3) the synthetic **anonymous** identity (priority 0, always tried first, no secret).
- **Secrets never logged** (§11.4.10): the table stores only `*_ref` handles; raw secrets live in `security/pkg/securestorage` (encrypted) or, on platforms with one, the OS keystore (see §2.6). Env-loaded secrets are read once, written to the secret store, and the env value is not echoed.

2.2 Discovery pipeline (anonymous-first, identity-fallback, remember)

```
[1 ENUMERATE HOSTS] every interface × method:
    mDNS/Bonjour (discovery/pkg/broadcast) ∪ NetBIOS/WS-Discovery
    ∪ ARP/subnet CIDR sweep
    (discovery/pkg/scanner) ∪ operator-configured static hosts
    | → set of reachable Host{ip, hostname, ifaces}
[2 ENUMERATE SHARES per host per protocol]
    SMB: discovery/pkg/smb (445/139) → share list (REAL
enumeration, NOT guessed §1.1)
    FTP/NFS/WebDAV: connect + list root (extend discovery with
per-protocol probers)
    | → set of ShareTarget{host, protocol, share/path,
port}
[3 PROBE AUTH – anonymous-first, then identities by priority]
    for each ShareTarget:
        try anonymous → if mount+ReadDir(".") OK (positive
evidence, smb_discovery.go:209–221 pattern) → BIND
        else for each enabled Identity ordered by priority:
            AuthScheme.Probe(...) → first success → BIND, stop
        else → record UNAUTHENTICATED (honest, no guess)
[4 REMEMBER]
    persist working ShareBinding{share_target, identity_id,
protocol, last_ok_at, probe_evidence_path}
[5 REGISTER + INGEST]
    upsert a storage_roots row per bound share (credentials via
identity ref, not inline)
    → UniversalScanner.ScanJob → AggregateAfterScan →
DuplicateDetection
```

Single-resource discipline (§11.4.119): probing is read-only and rate-limited (reuse discovery's semaphore); never write to a probed host. **Honest unreachable handling (§11.4.3/§11.4.6):** a host that does not answer is logged offline, never substituted with guessed shares (removes the `getCommonShares` guess path).

2.3 Persistence + reuse schema (new tables)

identities (§2.1 – secrets as *_ref handles only)
discovered_hosts (id, ip, hostname, first_seen,
last_seen, reachable, ifaces_json)
discovered_shares (id, host_id FK, protocol, share_name/
path, port, first_seen, last_seen,
enumeration_evidence_path)
share_identity_bindings (id, share_id FK, identity_id FK,
status ∈ {ok,unauthenticated,failed,revalidating},
anonymous_ok bool, last_ok_at,
last_attempt_at, probe_evidence_path,
UNIQUE(share_id, identity_id))
└ binding → storage_roots: storage_roots gains nullable
identity_id FK (replaces inline user/pwd over time;
migration keeps existing inline creds working, new shares
use identity_id – §11.4.124 no silent break)

- **Re-validation cadence (§11.4.128/§11.4.144):** a background follower re-probes each ok binding on a defined interval; on failure → revalidating → retry with the same identity, then re-run §2.2 step 3 (identity rotation). Drops logged honestly; never a silent stale “ok”.
- **Identity rotation/failure:** changing an identity’s secret invalidates dependent bindings → marked revalidating, not deleted (§11.4.124 investigate-before-remove).
- **storage_roots back-compat:** add identity_id INTEGER NULL + keep username/password columns during migration; resolver prefers identity_id when set, falls back to inline (so the existing main.go:377–416 SMB-root query path keeps working until migrated).

2.4 Catalog extension — register, dedup, group

1. **Auto-register:** each ok share_identity_binding upserts a storage_roots row (identity_id set, enabled=1). Existing UniversalScanner + AggregateAfterScan(storageRootID) (aggregation_service.go:60–67) ingest unchanged.
2. **Duplicate tracking across shares:** feed scanned files to DuplicateDetectionService (duplicate_detection_service.go). Dedup key precedence: **content hash** (DuplicateItem.FileHash) → **size + parsed-title + year/episode** fuzzy (existing Levenshtein/JaroWinkler stack) → external IDs. A title present on N shares becomes ONE MediaItem with N origins[] (each origin = {storage_root_id, share_id, path}).
3. **Grouping for browse:** MediaItem already groups files by title; extend its DTO with origins[] + duplicate_count + available_on_shares[] so the UI shows one card with a per-origin source picker (and a “duplicate” badge).

4. **Filter axes (new):** by host, by share, by protocol, by identity, by “has duplicates”, by online/offline source — all derivable from the new tables joined to `files.storage_root_id`.

2.5 UI/UX per app (OpenDesign tokens, light+dark, no overlap — §11.4.162)

Three new surfaces, each app:

- **A. Identity Manager** — list/add/edit/remove identities; type selector (credentials / api_token / ssh_key / ...); scheme-specific fields; **secrets masked** (show/hide), never rendered in plaintext logs or list views; priority ordering.
- **B. Discovered Shares** — grouped by host; per-share auth-status chip (anonymous / via <identity> / unauthenticated); “scan network” action; per-share enable/ingest toggle.
- **C. Grouped/deduped Browse** — existing catalog browse + duplicate badge + per-title origin/source picker + the §2.4 filters.

Per-app mapping: - **Android TV / phone (Compose, Material3):** new `IdentityManagerScreen`, `DiscoveredSharesScreen` under `ui/screens/...`; reuse OpenDesign-aligned `Theme.kt`. D-pad focus order for TV (mirror `LoginScreen.kt` focus pattern). Extend `CatalogizerApi.kt` (Retrofit) with the new endpoints. - **Web (React/Tailwind):** new pages under `src/pages/` + `IdentitiesApi` in `lib/api.ts`; **adopt OpenDesign tokens** (currently Tailwind-only — §11.4.162 gap to close). - **Desktop (Tauri/React):** extend `SettingsPage.tsx` SMB section into the full identity/shares model; adopt OpenDesign tokens. - **installer-wizard:** evolve `ConfigurationContext`’s `AccessSource` split (already present) into the identity model so first-run setup seeds identities.

Backend API (new, under /api/v1): GET/POST/PUT/DELETE /identities · POST /discovery/scan (kick host+share sweep) · GET /discovery/hosts · GET /discovery/shares · POST /discovery/shares/:id/probe (anon-first+identity-fallback) · GET /catalog/items?has_duplicates=&host=&share=&protocol=&identity= (extend existing browse). Bearer-auth + RBAC via `internal/auth/middleware.go`.

2.6 Security (§11.4.10 throughout)

- Secrets never in `storage_roots/identities` rows as plaintext → `*_ref` handles into `security/pkg/securestorage` (encrypted at rest), or OS keystore where available (macOS Keychain / Linux Secret Service / Android Keystore / Windows DPAPI) — **§11.4.81 cross-platform parity** with honest fallback to the encrypted file store.

- **Migrate the existing plaintext storage_roots.password** into the secret store as part of the migration (existing data is a current §11.4.10 debt).
- Secrets masked in all UIs; redact host/user via security/pkg/pii before any log line; .env stays gitignored; identity secrets never git-versioned; probe logs store evidence **paths**, not secret bytes.
- ssrf_guard.go (internal/services/ssrf_guard.go) gates any host/URL the probe touches.

3. Phased Implementation Plan (PWUs — §11.4.58)

Each Pwu ships four-layer coverage (§11.4.4(b)) + the §11.4.169 test-type matrix + captured physical evidence (§11.4.5/§11.4.69/§11.4.107).
 “Evidence” column = the anti-bluff artifact each test MUST capture. Audio/video N/A here; evidence is JSON probe results, DB row deltas, captured network traffic, screen renders.

PWU	Scope (files)	Test matrix (§11.4.169)	Mandatory captured evidence (anti-bluff)
P1 — Identity store + env loader	catalog-api/internal/identity/* (new), migration 00000X_identities, digital.vasic.config/pkg/env, security/pkg/securestorage wiring; consume CATALOGIZER_IDENTITY_*	unit (scheme parse, precedence, masking), integration (env→DB seed, secret store round-trip), security (no plaintext secret in DB/ logs — grep the row + log capture), stress (1000 identities), chaos (corrupt secret store → honest fail)	DB dump showing secret_ref not raised; secret; log capture proving redaction; test JSON; paired mutation (strip mask) → security test FA
P2 — Discovery/ probe engine	catalog-api/internal/discovery/* (new), reuse submodules/discovery/pkg/{scanner,smb}; replace smb_discovery.go:140–175 guess-fallback with honest SKIP	unit (host parse, anon-first ordering, identity fallback order), integration (probe against a containerized SMB/FTP/WebDAV booted via vasic-digital/containers §11.4.76/§11.4.161), e2e (scan→bind), full-automation (re-runnable §11.4.98), security (SSRF	captured probe result.json (per anonymous_ok / b identity / unauthenticated); or server-log proving real auth attempt; container boot log mutation (force guess fallback → test FA

PWU	Scope (files)	Test matrix (§11.4.169)	Mandatory capture evidence (anti-bl
P3 — Remember/persistence + revalidation	migrations discovered_hosts/ discovered_shares/ share_identity_bindings; storage_roots.identity_id; follower goroutine	guard, rate-limit), stress (concurrent multi-host), chaos (host drops mid-probe → §11.4.144 offline event) unit (binding state machine), integration (bind persists, re-probe path, rotation→revalidating), e2e (restart → bindings reused, no re-auth storm), stress (N bindings revalidate), chaos (SIGKILL mid-write → recover), concurrency/atomicity (UNIQUE(share,identity))	before/after DB snapshots; revalidation timeline log; mutation (drop UNIQUE → binding test FAILS)
P4 — Catalog source registration + multi-source dedup/group	bind→storage_roots upsert; UniversalScanner ingest; extend DuplicateDetectionService for cross-share; MediaItem.origins[] DTO	unit (origin merge, dedup key precedence), integration (same title on 2 container shares → 1 item, 2 origins), e2e (scan two shares → grouped), full-automation, stress (large corpus), benchmark (dedup p95 vs baseline)	result.json show item / N origins + confidence; query timing vs baseline mutation (break h first dedup → duplicate collapse test FAILS)
P5 — Backend API + RBAC	new /identities, /discovery/*, extended /catalog/items filters; internal/auth/middleware.go	unit (handlers), integration (auth required, RBAC), e2e (full REST flow), security (authz, injection, secret-leak in responses), DDoS/load (scan endpoint), API-readiness	HTTP transcript (request/response secrets masked); 401/403 negative mutation (drop authentication security test FAILS)
P6 — App UI: Identity Manager + Discovered Shares + Grouped Browse	per-app screens §2.5; OpenDesign tokens for web/desktop/phone	unit (components), UI-driven (§11.4.48), host-side rendered-pixel visual proof (§11.4.170 Roborazzi/Paparazzi/Playwright) light+dark + OCR layout oracle ,	rendered PNG per screen×state×{light+golden-diff + OC overlap verdict; live device recording (§11.4.159) MP4 + read; mutation (ov

PWU	Scope (files)	Test matrix (§11.4.169)	Mandatory capture evidence (anti-bl
P7 — Wiring + full integration + HelixQA + Challenges	end-to-end glue; Challenge banks; HelixQA autonomous session	e2e, full-automation (§11.4.143 real journey), accessibility (WCAG-AA) Challenges (anon-first + remember + dedup), HelixQA full session, full-suite retest (§11.4.40), chaos+stress aggregate	inject → visual gar FAILs) HelixQA result.j per bank; recorded autonomous sessi §11.4.116 sync-ch event stream

Cross-cutting per PWU: §11.4.146 reproduce-first RED on the broken/absent state → same-test GREEN → extend-to-all-cases; §11.4.150 deep multi-angle research footer; §11.4.125/§11.4.142 independent review to clean GO (§11.4.134); §11.4.135 regression guard registered on every closure.

3.1 Operator-gated items (flag honestly, §11.4.3/§11.4.21)

- **Real LAN NAS to probe with the provided identities** — autonomous CI uses **containerized** SMB/FTP/NFS/WebDAV servers (§11.4.76) for deterministic proof; a **real Synology/NAS sweep** with CATALOGIZER_IDENTITY_* is operator_attended (needs the physical LAN + real hosts). NOT a faked PASS — a tracked migration item.
- **OS keystore on each platform** (Keychain/Secret Service/Android Keystore/DPAPI) — may need per-platform operator setup; honest fallback to encrypted file store meanwhile (§11.4.81).
- **NFS on macOS/Windows** — build-tag-gated today (nfs_client_darwin.go/_windows.go); per-OS parity needs validation (§11.4.81).

4. Open Questions for the Operator (§11.4.66)

1. **Protocol scope, first cut:** SMB is the only protocol with a discovery path today (smb_discovery.go). Do we ship **SMB-first** (P1–P5 SMB-only, then fan out FTP/NFS/WebDAV in P6+), or build all four probers up front? (Recommended: SMB-first — fastest to real evidence, lowest blast radius.)
2. **Real hosts on the LAN:** Is there a real Synology/NAS (or other SMB/NFS host) reachable on this LAN to probe with the CATALOGIZER_IDENTITY_* identities for §11.4.143/§11.4.159 live

- evidence, or do we rely solely on containerized servers for autonomous proof and mark real-NAS as operator_attended?
3. **Secret store:** OS keystore (Keychain/Secret Service/Android Keystore/DPAPI) per platform, or the security/pkg/securestorage encrypted file store as the single cross-platform default? (Recommended: encrypted file store as default, keystore as opt-in per §11.4.81.)
 4. **Existing plaintext storage_roots.password:** confirm we migrate it into the secret store and null the plaintext column (one-time data migration, §11.4.10 debt repayment) vs. leave inline-cred shares as legacy.
 5. **Anonymous-first safety:** any hosts where an anonymous mount attempt is undesirable (e.g., would trip an IDS/lockout)? Should anonymous probing be opt-out per host?
 6. **OpenDesign rollout:** web/desktop/phone are Tailwind/Material-only today — confirm closing the §11.4.162 token gap is in-scope for this capability (Recommended: yes, P6).
-

5. Evidence Index (file:line anchors used above)

- Identity gap: .env.example:46–65 (NAMES only) + zero *.go consumers
- Share model + creds: catalog-api/models/file.go:48–70; DB database/migrations/000001_initial_schema.sqlite.up.sql:4–26
- Existing SMB discovery + guess-fallback: catalog-api/internal/services/smb_discovery.go:57–95, 140–175, 178–225
- Discovery routes: catalog-api/main.go:1267–1271; multicast announce main.go:771–795; SMB root query main.go:377–416
- Protocol clients: catalog-api/filesystem/interface.go:1–16, filesystem/factory.go:15–50, filesystem/smb_client.go:38–73
- Scanner: catalog-api/internal/services/universal_scanner.go:19–67
- Aggregation: catalog-api/internal/services/aggregation_service.go:60–67
- Dedup: catalog-api/internal/services/duplicate_detection_service.go:15–90
- Creds read path: catalog-api/handlers/scan_handler.go:294–308, handlers/copy.go:326
- Owned submodules: submodules/{discovery,auth,filesystem,security,database,config}/go.mod
- App UIs: installer-wizard/src/components/wizard/SMBConfigurationStep.tsx:26–478, installer-wizard/src/contexts/ConfigurationContext.tsx; catalogizer-desktop/src/pages/SettingsPage.tsx:24–114; catalogizer-web/src/pages/Settings.tsx:34–101 + src/lib/api.ts; catalogizer-androidtv/...

ui/screens/login/LoginScreen.kt:224–270,538–583 + ui/theme/
Theme.kt + data/remote/CatalogizerApi.kt:14–49

6. Sources verified (§11.4.150 — deep-research note)

Design synthesized from in-repo evidence (above) + owned-submodule capabilities. External protocol/standard cross-references (SMB share enumeration via srvsvc/IPC\$, mDNS/WS-Discovery host enumeration, anonymous-bind semantics, OS keystore APIs) are **flagged for the per-PWU §11.4.150 deep-research pass** before implementation — this is a design/survey doc, not a closure; no implementation decision here is final until that pass cites latest authoritative sources. NO external solution adopted yet — original composition of owned submodules.